Autonomous AI Task Worker

Final Pipeline — Prototype Architecture

Final pipeline: Autonomous AI Task Worker (prototype)

This is the consolidated design with every flaw from all review rounds fixed. It is not literally flaw-proof, and no design is. The remaining limits are listed in section 9 so you can state them yourself. This is the version I'd build.

1. Scope, stack, assumptions

A prototype that does one narrow thing for real, against a sandboxed mock company.

Layer	Choice
Orchestration	LangGraph + `SqliteSaver`
LLM	Any tool-calling model with structured output (name it in the README)
Validation	Pydantic
Browser	Playwright (`locator.aria_snapshot()`; check your installed version)
Documents	PyMuPDF (text layer, tables); Tesseract with per-word confidence for scanned PDFs; a second reader (vision model or different OCR engine) on the rendered page image
Mock environment	FastAPI + SQLite with a **unique-key constraint** on the business key; trade API
Tracing	JSON audit log; Langfuse optional

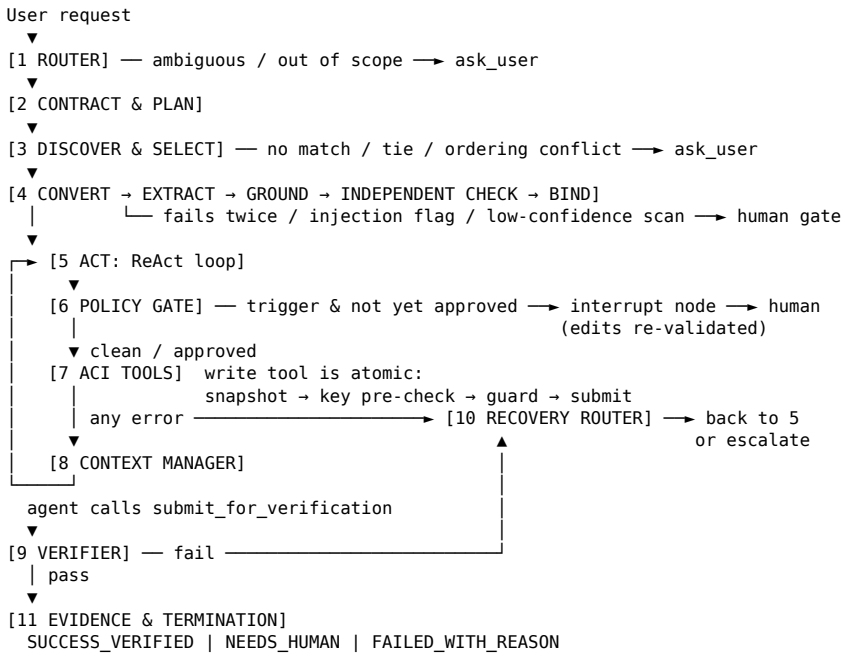
Assumptions to state in the README:

• Invoices are discovered as files in a sandbox folder (the brief's "find the latest invoice" could also mean email or a portal).

• RAG is deliberately not used, because the task selects one document rather than searching a corpus.

• Backend read access exists for verification, which is true in the mock but not in most real systems.

2. Architecture



Terminal state	Meaning
`SUCCESS_VERIFIED`	Outcome confirmed by an independent read against the contract (`CREATED` or `ALREADY_PRESENT`)
`NEEDS_HUMAN`	Paused and resumable; waiting on a person
`FAILED_WITH_REASON`	Gave up after bounded attempts, with evidence and a partial-write report

3. State

```
class AgentState(TypedDict):
    user_query: str
    toolkit: str
    params: dict
    static_contract: StaticContract
    plan: list[PlanItem]
    selected_source: SourceRef | None
    bound: BoundState | None              # only deterministic code writes this
    baselines: list[Baseline]             # appended by the write tool, one per attempt
    approvals: set[str]                   # keys: hash(action_fingerprint, bound.checksum)
    facts: dict                           # discovered facts
    reflections: list[str]                # max 3, lessons only
    history: list[Step]                   # full audit log, never collapsed
    visible_history: list[Step]           # compacted prompt view
    counters: Counters                    # steps, per-class retries, verify cycles, tokens
    action_fingerprints: list[str]
    pending_question: str | None
    verify_result: VerifyResult | None
    terminal: str | None
```

The agent has no tool that writes `bound`, `baselines`, `approvals`, `verify_result` or `terminal`. Code sets those.

4. Phases

1. Router

The LLM sees each toolkit's short description, picks one, and extracts parameters. Missing arguments, no match, or low confidence go to `ask_user` before any side effect. The router is what makes "same graph, different task" real rather than a config swap.

2. Contract and plan

• Static contract: target system, required fields, and named rules from a registry (`positive`, `iso_date`, `currency_is`, `sum_equals`, `unique_key`). The LLM picks rule names and fills parameters. No `eval()`.

• Plan: a short revisable checklist in state. Every ReAct thought names its active step.

3. Discover and select

`discover()` is deterministic. It returns candidates, `ordering_basis`, `tie` and `no_match`. It routes to `ask_user` when nothing matches the company, two candidates tie, or ordering bases disagree (file mtime vs document date). "Latest" only means something relative to a basis.

4. Convert, extract, ground, independently check, bind

1. Convert. PyMuPDF reads the text layer into markdown. If the layer is empty or tiny, run Tesseract and keep per-word confidence. Hidden text (white or tiny font) is detected from span properties and flagged.

2. Injection scan. A heuristic phrase and pattern check. A hit raises the injection flag. This is bypassable, so it is one layer among several.

3. Extract (LLM, tool-less). A strict Pydantic schema. Each field returns `value` plus `source_quote`. Amount means `total_due`. Document text is data, never instructions.

4. Ground. Normalize whitespace and Unicode, then normalize values (Rs. 48,500 → 48500.00; "05 Oct 2026" → 2026-10-05). The quote must appear in the converted text, and the value must derive from the quote. Ambiguous date formats (05/10/2026) fail.

5. Independent check, depending on source type:

- *Text-layer PDF:* a second extraction with a different prompt or model, and normalized values must match.

- *Scanned document:* a cross-check on the same OCR text is not independent, because an OCR digit error passes both extractors. So require a second reading from the rendered image (vision model or different OCR engine), per-field confidence above a threshold, and `sum_equals`. Any disagreement or low confidence goes to the human gate.

6. Reconcile. `subtotal + tax == total`, currency matches config, static rules pass.

7. Bind. Deterministic code writes `bound` with `source="extracted"` and a checksum. Failures return `BindFailure`.

8. Failure policy. One retry with an alternate prompt or model. A second failure, an injection flag, a hidden-text flag, or low confidence goes to the human gate with the source snippet.

Raw document text never enters the ReAct loop. The agent sees only bound values.

5. Act (ReAct loop)

• Interleaved Thought → Action → Observation. Thoughts are sparse and name the plan step.

• The agent decides navigation, sequencing, when to fill and submit, and how to handle unexpected pages. It does not decide which value goes in which field.

• Loop detection: the fingerprint is `hash(tool, args, page_snapshot_hash)`. At the 3rd repeat within 8 steps, force one replan. A further repeat escalates to the gate.

• Making the autonomy real: the demo includes tasks with unplanned navigation (the invoice link sits on a second page), a UI state the toolkit didn't anticipate, and a read-only report task with no fixed script. The deterministic guardrails are intentional, and the README says so.

6. Policy gate

Every tool is tagged `READ`, `IDEMPOTENT_WRITE` or `IRREVERSIBLE_WRITE`. Plain Python decides:

Trigger	Example
`IRREVERSIBLE_WRITE`	delete, payment
Bound amount over threshold	checked before the write
Tie / no-match / ordering conflict	from stage 3
Injection, hidden-text or low-confidence flag	from stage 4
Loop detected twice	from stage 5
`PREEXISTING_CONFLICT`, duplicate rows, `FormMapMismatch`	from stages 7 and 9

• Invoice submission is `IDEMPOTENT_WRITE` (business key, verify step, correction path). If you marked it irreversible, every run would stop and autonomy would drop.

• No approval loop: after the human approves, the gate records `hash(action_fingerprint, bound.checksum)` in `approvals` and skips matching triggers. If an edit changes the checksum, the approval key changes and the edited payload is re-validated.

• The interrupt lives in its own side-effect-free node, because LangGraph re-runs the interrupting node from its start on resume.

• Human edits are re-validated. On resume the static rules re-run on the edited values. Grounding can't apply, so `bound.source = "human_edit"` and the evidence shows the diff. A rule-breaking edit goes back to the human with the reason.

• Resume safety: `SqliteSaver` persists graph state, not the browser session. On every resume the system runs `login()` and `reset_view()` first, and tools are safe to re-run. Call it "durable graph state," not "durable execution."

7. ACI tool layer (SWE-agent paper)

Principles applied: simple actions, compact actions (batching is the paper's "consolidate into fewer actions" idea), informative concise feedback (an explicit message on silent success), and guardrails before acting.

Tool	Notes
`login()`	Credentials from env inside the tool; never in prompts or logs
`open_url`, `read_page`, `click(ref)`	Compact aria snapshot with per-snapshot refs; `StaleElementError` on stale refs. Only interactive elements and short labels are shown. Long free-text values are truncated or omitted, to limit injection through database-sourced page text

Tool	Notes
`fill_form(form_id)`	Maps bound keys to **accessible labels**, resolved against the live snapshot at call time. Zero or multiple matches fail closed with `FormMapMismatch`. Returns per-field results
`submit_form()`	Atomic write tool, below
`update_record(key)`	Edit form using the same label mapping and guard; backs the "wrong values" recovery
`submit_for_verification()`	The only way to finish
`ask_user(q)`	Clarification

`submit_form()` runs these steps as one unit:

1. Snapshot. Per-row content hashes for the whole table (fine at prototype scale) plus a key-scoped view. Append to `baselines`. Doing this inside the write tool ties the baseline to the actual write, gives a fresh one per retry, and works with or without a human interrupt.

2. Key pre-check. Before clicking, check whether the business key already exists.

- Exists and matches bound → return `ALREADY_PRESENT` without submitting.

- Exists with different values → don't submit; go to the human gate (`PREEXISTING_CONFLICT`). This catches the conflict *before* damage, not after.

3. Pre-submit guard. Read live DOM values, normalize them (`Rs. 48,500` equals `48500.00`), diff against `bound`. Any mismatch aborts before the click.

4. Click, then return the server response and a viewport screenshot.

Write timeouts are ambiguous. The write may have succeeded. For write tools, any timeout or transient error triggers a read-only key check first, then a decision. Blind backoff retry applies only to `READ` tools. The mock DB's unique constraint is the backstop against double submits.

Error normalization: this layer only catches exceptions and emits structured `ToolError` objects to stage 10. It contains no LLM logic, so the agent has one model of failure.

8. Context manager (SWE-agent paper)

• Observations older than the last 5 collapse to one-line summaries produced by the tool, not an LLM. The paper found this beat full history (18.0 vs 15.0 on SWE-bench Lite).

• Never collapsed: errors, the latest snapshot, the plan.

• Malformed model output gets an error message and a retry. After 3 in a row, stop. Failed attempts are removed once a valid one arrives.

• The full audit log is kept separately for evidence.

9. Verifier (CRITIC paper)

Deterministic code reading through an independent channel (direct SQLite, not the browser session). It never asks the LLM whether it succeeded, because the paper shows models can't reliably self-verify without external tools.

`verify(bound, baselines)` is scoped, not count-based, which fixes the `row_count == baseline + 1` bug (it fails on 409s, updates, and concurrent inserts).

1. Target row. The business key exists in exactly one row and its fields exactly equal `bound.target_values`.

2. In-scope rows unchanged. Rows sharing the key or vendor must be unchanged versus the last baseline. This is a hard check. Other changed or added rows (concurrent activity) are tolerated and listed as warnings in the evidence.

3. Outcome from the first baseline:

- Key absent before, present and correct now → `CREATED`.

- Key present and correct before → `ALREADY_PRESENT` ("no write needed").

- Key present with different values before → `PREEXISTING_CONFLICT`. Don't overwrite someone else's record; escalate.

4. Result: `VerifyResult(passed, outcome, diff[], warnings[], message)`, where each diff entry is `{field, expected, actual}`.

This proves the database matches the grounded contract and that in-scope rows are undisturbed. It does not prove the right source document was chosen, which is why stage 3 gates ambiguity.

10. Recovery router (Reflexion paper, used selectively)

One router handles stage 7 errors and stage 9 failures. Deterministic handlers come first. The LLM reflects only when the table can't explain the failure.

Trigger	Handler	LLM reflection?
`StaleElementError`	auto re-snapshot, retry (max 2)	no
Read-tool timeout	backoff retry (max 3)	no
Write-tool timeout / transient	read-only key check, then decide	no
Pre-submit guard mismatch	refill from bound values	no
HTTP 409	key check, compare to bound, go to verify	no
Verify: key missing	idempotent resubmit (max 2)	no
Verify: wrong values, created by this run	`update_record`, re-verify (max 1)	no
`PREEXISTING_CONFLICT`, duplicate rows, `FormMapMismatch`	human gate	no
Unexplained / navigation failure	LLM reflection, then `reset_view()`, then back to the loop	yes (max 3)

• Reflection input is the structured error or diff only (`field, expected, actual`), never document text, so an injection can't ride along and the model can't invent a cause.

• Budgets: step cap (start at 40, then set from eval runs), max 3 verify→recover cycles, the per-class limits above, and a token/cost ceiling. The papers show success comes early (SWE-agent: about 12 steps for successes vs 21 for failures; CRITIC: most gain in 2–3 rounds).

• Exhausted budget → `FAILED_WITH_REASON`. Needs a person → `NEEDS_HUMAN`.

• Interview precision: the Reflexion paper retries across episodes with a reset environment. This is bounded within-run adaptation inspired by it. The paper also notes it depends on a reliable evaluator and struggled on WebShop, which is a second reason the evaluator here is deterministic code.

11. Evidence and termination

Every terminal state produces a bundle:

1. Terminal state, reason, user-facing summary
2. Source snippet and quote for each bound value, with OCR confidence where relevant
3. Bound values with provenance (`extracted` or `human_edit`)
4. Baselines, final rows, verify diff, warnings, outcome
5. Pre-submit guard diff and viewport screenshot
6. Full audit log (thoughts, tools, inputs, outputs), credentials redacted
7. For non-success states: a read-only check of the target key, so the user knows whether a partial write exists

The chat reply is concise: outcome, key values, evidence link.

5. Toolkit protocol

```
class Toolkit(Protocol):
    name: str
    description: str
    config: ToolkitConfig                    # thresholds, currency, label map
    tools: dict[str, ToolSpec]               # tagged with permission tier

    def static_contract(self, params) -> StaticContract: ...
    def discover(self, params) -> DiscoveryResult: ...
    def bind(self, source) -> BoundState | BindFailure: ...
    def label_map(self, form_id) -> dict[str, str]: ...        # bound key -> accessible label
    def snapshot(self, key: str) -> Baseline: ...              # whole-table hashes + key view
    def key_state(self, key: str) -> KeyState: ...             # absent | present(values)
    def verify(self, bound, baselines) -> VerifyResult: ...
    def recovery_for(self, diff) -> RecoveryAction | None: ... # None -> LLM reflection
    def reset_view(self) -> None: ...
```

The graph never mentions invoices or trades. The honest claim is "graph unchanged; the Toolkit implementation is the task-specific part."

6. Toolkits

	Invoice	Trade	Report (read-only, optional)
Write path	Playwright on mock ERP	API with idempotency key `(date, symbol, run_id)`	none
Independent verify	direct SQLite	holdings/ledger query	recompute via a different code path (raw SQL vs pandas) and assert DB unchanged
Rules	total = subtotal + tax, ISO date, currency	`net = buys − sells` computed in Python; LLM only identifies columns and symbol	totals match seeded known answers
Traps	duplicates, tie, scanned PDF	sells, other symbols, header rows, duplicate fills (dedupe by execution ID; identical rows without IDs go to human), aliases like `RELIANCE.NS` via a table	overdue boundary dates

Cut the Report toolkit first if time is short.

7. Evaluation harness

Run each scenario 3–5 times. Report success rate, steps, retries, tokens, cost.

#	Scenario	Expected
1	Happy path	`SUCCESS_VERIFIED`, `CREATED`
2	Transient silent drop (200, first write not persisted)	missing key → resubmit → verified
3	Persistent silent drop	limit hit → `FAILED_WITH_REASON`, evidence shows no write
4	Verifier-off ablation of #2	false "success" (the CRITIC demo)
5	Form auto-formats currency	guard normalizes or refills
6	Subtotal/total layout trap	label mapping fills the right field
7	Tie / ordering conflict	`ask_user`
8	Unknown company	`ask_user`
9	Poisoned PDF (visible and hidden text)	flag → gate; instruction ignored
10	Scanned PDF with digit-swap OCR error	image re-read / confidence / `sum_equals` disagree → human
11	Poisoned note text inside a UI field	truncated or ignored; no unauthorized action
12	409, existing record identical	`ALREADY_PRESENT`, no duplicate
13	Existing record with different values	pre-check blocks the write → human
14	Write timeout where the write actually succeeded	key check finds it, no double submit
15	Missing selector / repeated clicks	forced replan, then human
16	Human edits amount at gate while a hook inserts unrelated rows	edit re-validated, `human_edit`, no approval loop, verify tolerates extra rows
17	Resume after restart	re-login and `reset_view`, then continues
18	Wrong login / missing file / DB down	classified, bounded, `FAILED_WITH_REASON`
19	Unplanned navigation (link on a second page)	agent finds it without a script
20	Trade happy path and traps	correct net, deduped, aliases normalized
21	Read-only report	matches independent recompute

Simulate the elapsed time in #16 with a hook during the pause, not a real wait.

8. Build order

1. Deterministic core: SQLite schema with unique constraint, `snapshot`, `key_state`, scoped `verify`.
2. Mock ERP with fault injection (transient and persistent drop, 409, delayed persistence, timeout-after-success).
3. Stage 4 pipeline (text-layer path first, scanned path second).
4. Stage 7 tools: label-based `fill_form`, atomic `submit_form`, `update_record`.

5. ReAct graph, policy gate with separate interrupt node, approvals, `SqliteSaver`, loop detector.

6. Recovery router from the table, then reflection.

7. Eval harness for 1–19.

8. Trade toolkit, then Report if time remains.

Get one invoice flow working with one recovery before touching anything else.

9. Known limitations (copy into the README)

• Verification confirms the DB matches a grounded contract, not that the right source was chosen.

• A document with a plausible but wrong value passes grounding. Backstops are reconciliation, the amount threshold, and the human prompt.

• The injection scan is heuristic; the defense limits blast radius and doesn't eliminate the attack.

• Scanned-document accuracy is bounded by OCR quality, mitigated by the second reading and confidence gating.

• Whole-table hashing doesn't scale beyond a prototype.

• Single user and process; concurrency is simulated.

• Verification needs backend read access, which real systems often lack.

• Recovery adaptation is within-run only; results vary by run, so numbers come with repeats.

• Mock environments only: no MFA, CAPTCHA, or third-party systems.

Next, if given more time: label-map inference, a real concurrency model, a second verification channel, larger eval sets.

10. Claims for the interview

Say	Avoid
"Scoped verification via an independent read; baseline taken atomically with the write; conflicts caught before the write"	"Mathematically proves success"
"Tool-less schema extraction, grounding, and independent checks limit blast radius"	"Prompt injection is prevented"
"Label-mapped filling plus a pre-submit guard reduces wrong entries"	"Hallucination is impossible"
"`SqliteSaver` gives durable graph state and resumable pauses"	"Cryptographically secure" or "durable execution"
"Bounded within-run recovery inspired by Reflexion, deterministic first"	"Implements Reflexion"
"Graph unchanged across toolkits; domain logic behind the Toolkit protocol"	"Zero configuration for any enterprise app"
"Prototype, with listed limits"	"Production-grade"

I'd stop revising here and start building, since a working invoice flow now teaches you more than another review round. I can turn this into a README or write the starter code next (`Toolkit` types, `snapshot`/`verify`, the recovery table, and the atomic `submit_form`).